

CURLIX

Product Requirement Document (PRD) v2.0 — Anonymous Free Stack MVP

A scalable, production-grade URL shortening platform engineered with Redis caching, async analytics queues, anonymous ownership tokens, and abuse protection — built entirely on zero-cost infrastructure.

1. Product Name

Product Name: Curlix

Tagline: Shorten smarter. Track everything.

Version: 2.0 — Anonymous MVP

Document Type: Product Requirement Document (PRD)

Infrastructure Goal: 100% Free Tier Compatible

2. Target Users

- Developers needing fast API-based URL shortening
- Students learning backend engineering and system design
- Content creators needing lightweight analytics
- Recruiters/interviewers reviewing scalable backend projects

3. Problem Statement

Long URLs are difficult to share and provide no tracking visibility. Most URL shorteners hide analytics and advanced features behind paid plans. Existing tutorial projects rarely demonstrate real backend engineering concepts such as: Redis caching, queue-based processing, async analytics, and abuse protection. Curlix solves this by providing a fully free, scalable, engineering-first URL shortening system.

4. Core Features

4.1 URL Shortening

- Create short URLs using nanoid-based short codes
- Optional custom aliases
- Optional expiry timestamps
- QR code generation

4.2 Redirect Engine

- Redis-first lookup for ultra-fast redirects
- Cache miss fallback to PostgreSQL
- 302 redirects for analytics accuracy

4.3 Analytics System

- Queue-based analytics using BullMQ
- Device, timestamp, referrer tracking
- Click charts and aggregated statistics

4.4 Anonymous Ownership

- No signup or login required
- Owner token generated during link creation
- Token required for delete/edit operations

5. User Flow

Flow A — Create Link

1. User submits long URL
2. CAPTCHA verification succeeds
3. Backend validates URL
4. nanoid short code generated
5. Owner token generated and hashed
6. URL saved in PostgreSQL
7. Redis cache primed
8. Short URL + owner token returned

Flow B — Redirect

1. Visitor accesses short URL
2. Redis checked first
3. On miss → PostgreSQL lookup
4. Analytics job pushed to queue
5. Redirect returned instantly

Flow C — Analytics

1. User opens analytics page
2. Analytics API fetches aggregated stats
3. Charts rendered in frontend dashboard

6. Technology Stack (100% Free)

Layer	Technology	Free Service
Frontend	React + Vite	Vercel Hobby
Backend	Node.js + Express	Render Free
Database	PostgreSQL	Supabase Free
Cache	Redis	Upstash Free
Queue	BullMQ	Uses Upstash Redis
Bot Protection	Cloudflare Turnstile	Free
Keep Alive	UptimeRobot	Free
Code Hosting	GitHub	Free

7. Security Requirements

7.1 CAPTCHA Protection

- Cloudflare Turnstile blocks automated bots

7.2 Rate Limiting

- Redis-backed request limiting
- Shared across multiple instances

7.3 Owner Token System

- Random owner token generated during creation
- Token hashed before DB storage
- Required for edit/delete actions

7.4 Input Validation

- Reject localhost and private IP URLs
- Alias sanitization and reserved keyword blocking

7.5 Infrastructure Security

- Helmet.js security headers
- HTTPS enforced automatically
- Environment variables protected

8. Scaling Requirements

- Redis used as L1 cache for redirect lookups
- Queue-based analytics prevent redirect slowdowns
- Stateless backend enables horizontal scaling
- PostgreSQL indexing optimized for analytics queries
- Redis cache invalidation on updates/deletes
- Worker processes can scale independently

9. Database Schema

Column	Type	Purpose
id	uuid	Primary key
short_code	varchar(30)	Unique short URL code
long_url	text	Destination URL
owner_token_hash	text	Hashed ownership token
created_at	timestamptz	Creation timestamp
expires_at	timestamptz	Optional expiry
is_active	boolean	Soft delete flag

10. API Endpoints

POST /api/shorten → Create short URL

GET /:code → Redirect to destination URL

GET /api/analytics/:code → Fetch analytics

DELETE /api/links/:code → Delete link using owner token

Document prepared for the Curlix Engineering MVP — May 2026